Report Generator Pro

Sales Analytics & Automated Reporting Platform

A fully offline PyQt5 desktop application that converts structured sales data into business insights and branded Excel and PDF reports.

Version 1.0 · Python 3.10+ · Windows

Technology stack: Python · PyQt5 · openpyxl · ReportLab · PyMuPDF · pytest · JSON · PyInstaller

110 automated tests passing · end-to-end verified · benchmarked to 10,000 records

Project Overview

What the project is. Report Generator Pro is a professional desktop application for Windows that turns structured sales data into business insights and branded Excel and PDF reports. It offers a spreadsheet-style data editor, Excel import with automatic header detection, an analytics dashboard with charts and filters, a section-based report builder, in-app document previews, generation history, themes, and full offline operation.

What problem it solves. Producing recurring sales reports from raw data is slow and error-prone when done by hand in Excel. This application automates the whole chain — validation, aggregation, insight generation and document production — into a few clicks, with consistent branding and no cloud dependency.

Who will use it. Small-business owners, sales analysts and back-office staff who keep sales records in spreadsheets and need repeatable, professional reports without manual formatting work.

Main objective. Deliver a complete, production-quality desktop product — not a prototype — with clean modular architecture, thorough automated testing, honest error handling, and packaging that a non-developer can run.

Key Features

Feature	Description
Data entry	Spreadsheet-style inline editor; 10 columns per record; computed Revenue / Net Revenue columns are read-only; duplicate and clear row actions
Excel import	.xlsx import with automatic header detection and flexible column mapping — works with or without an ID column
Dashboard	5 KPI cards, 4 live charts, filters (period / region / category / product), period comparison, recent activity
Analytics	Revenue and units by month, category and region; top/bottom products; contribution percentages; best/worst periods
Insights	Deterministic, data-backed statements — never fabricated; changes below 1% are suppressed as noise
Report builder	8 selectable sections; per-report branding (company name, colours, fonts, footer text)
Excel/PDF generation	Branded workbooks (real number formats, freeze panes, auto-filter, charts) and multi-page PDFs with headers/footers and page numbers
Preview	In-app preview of generated PDFs (rendered pages) and Excel sheets before opening externally
History	Every generation recorded with status, record count, file paths and duration; searchable and filterable
Settings	Output directory, theme, branding, history limit — persisted to the user configuration directory
Themes	Light and dark themes via QSS stylesheets, applied application-wide and persisted
Accessibility	Keyboard shortcuts (Ctrl+1–8), tooltips, visible focus indicators, icon + colour status signals

Technology Stack

Technology	Role in the project
Python 3.10+	Application language — dataclasses, type hints, pathlib, tempfile
PyQt5	GUI framework: views, model/view tables, QThread workers, signals/slots, QSS themes

Technology	Role in the project
openpyxl	Excel generation and import (native charts, number formats, conditional formatting)
ReportLab	PDF generation (platypus layout engine, TTF font registration)
PyMuPDF (fitz)	Renders PDF pages to images for the in-app preview
pytest	Automated test suite — 110 tests
JSON	Configuration and history persistence (atomic writes with backups)
PyInstaller	Standalone Windows executable packaging
Git / GitHub	Version control, .gitignore, README, requirements.txt

System Architecture

The application follows a layered architecture. Each layer only talks to the layer below it, and UI events travel upward through Qt's signals/slots mechanism:

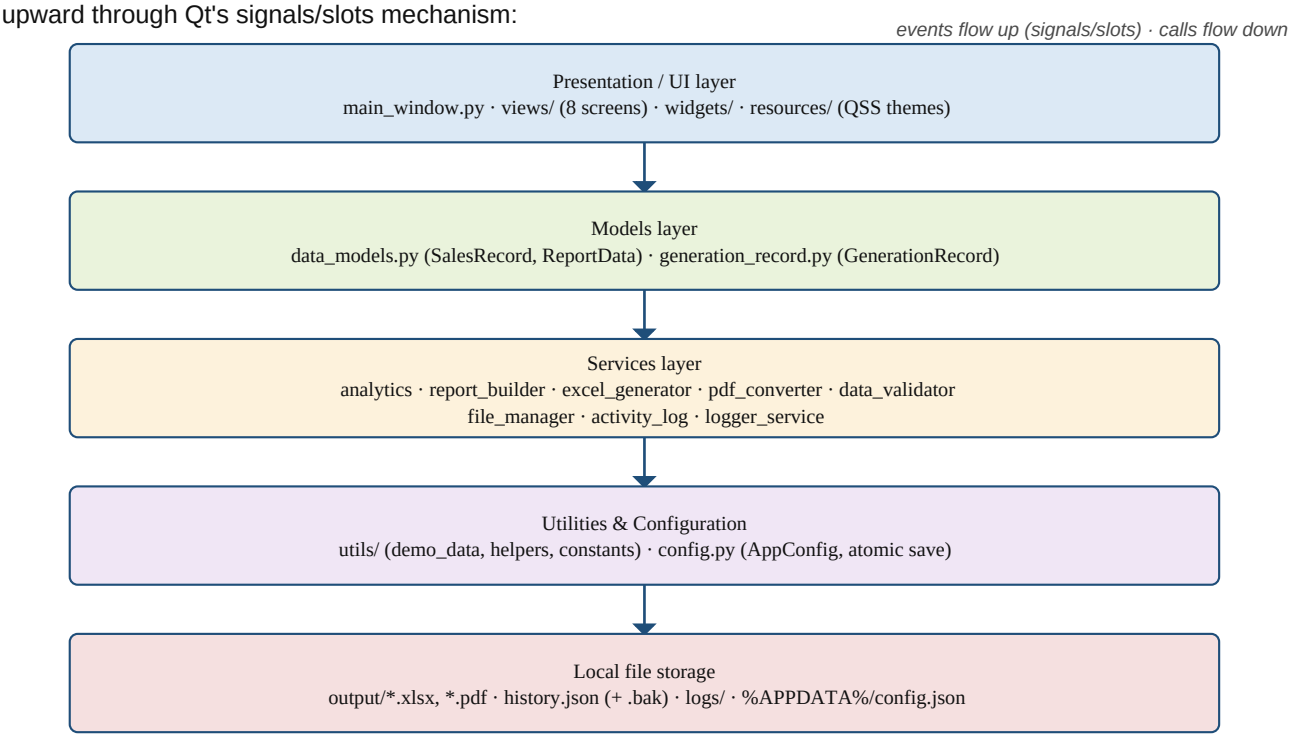


Figure 1 — System architecture. The UI layer never touches files or calculations directly: it calls services, services use models and config, and only the lowest layer reads/writes disk. This keeps the codebase testable — services can be tested headlessly without any GUI.

Project Folder Structure

main.py is the only top-level script; every feature lives in a focused module under app/. Important files and their responsibilities:

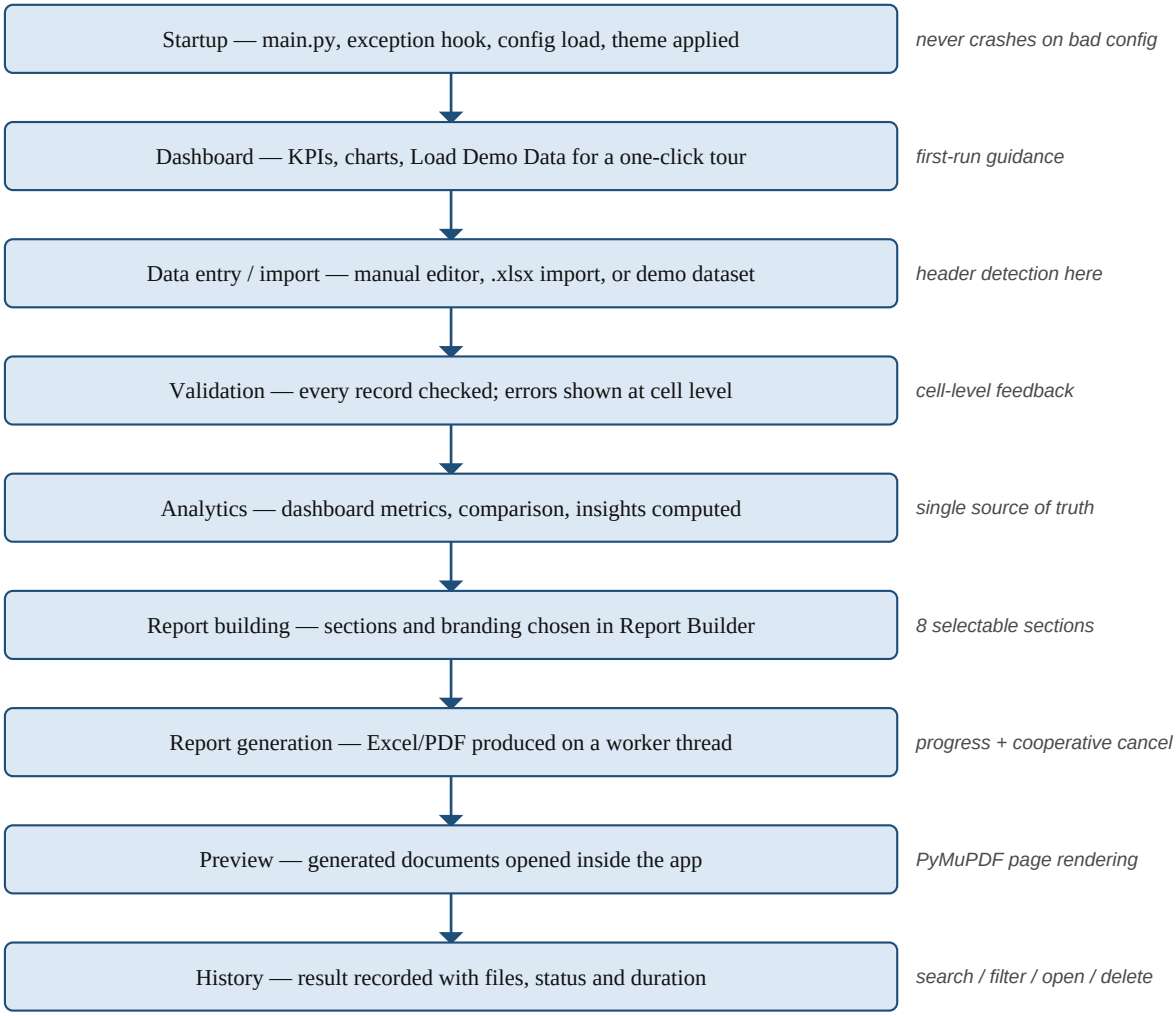
```
pyqt-app/
■■■ main.py entry point; installs the global exception hook
■■■ requirements.txt pinned dependencies
■■■ build_windows.bat one-command PyInstaller build
■■■ report_generator_pro.spec PyInstaller spec (resources + fonts)
■■■ README.md usage, architecture and testing documentation
■■■ app/
■ ■■■ main_window.py sidebar navigation, screen wiring, shortcuts, themes
```

```
■ ■■■ config.py AppConfig dataclass, atomic JSON persistence, defaults
■ ■■■ views/ 8 screens (home, data entry, builder, comparison,
■ ■ report center, history, settings, about)
■ ■■■ widgets/ BarChart, DropZone, DocumentPreview (reusable)
■ ■■■ models/ SalesRecord, ReportData, GenerationRecord
■ ■■■ services/ analytics, report_builder, excel_generator,
■ ■ pdf_converter, data_validator, file_manager,
■ ■ activity_log, logger_service
■ ■■■ utils/ demo_data, helpers (imports), constants
■ ■■■ resources/ light.qss, dark.qss, theme_tokens.py, fonts
■■■ tests/ test_all.py, test_analytics.py, test_builder.py,
■ conftest.py (isolated env), verify_integration.py,
■ benchmark.py
■■■ sample_data/ sales_sample.csv / .xlsx (120-record demo dataset)
```

Each services/ module has a single responsibility, which is what makes the system easy to extend and test: analytics.py owns every calculation, data_validator.py owns every rule, file_manager.py owns every path, and so on.

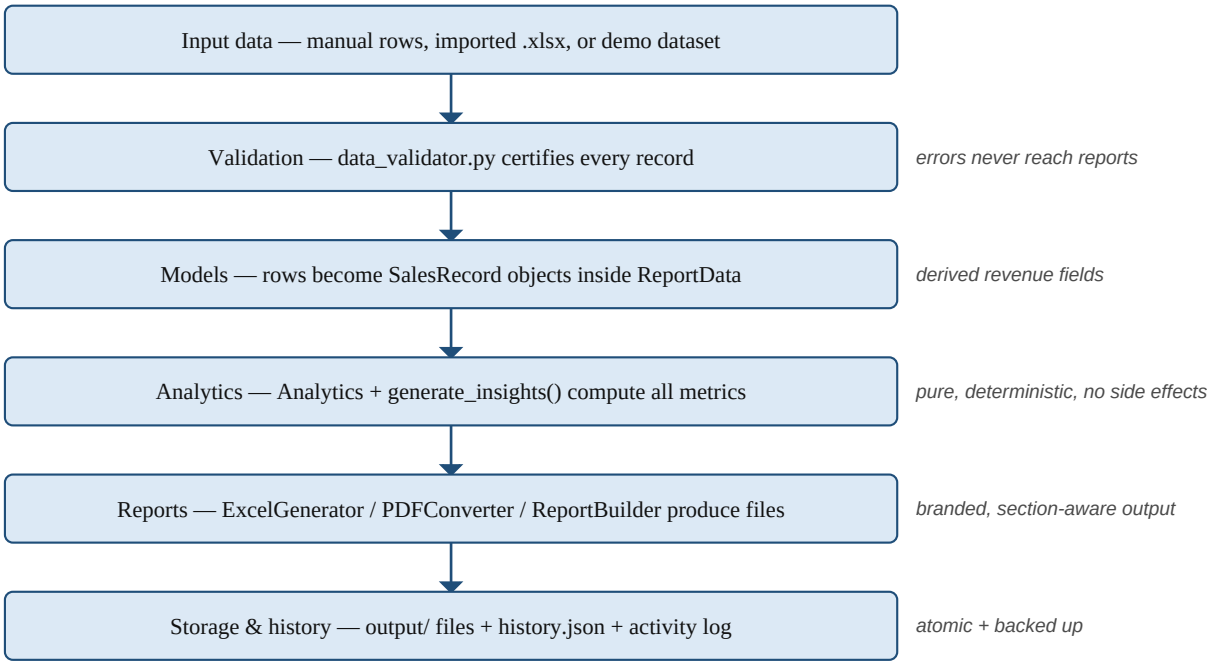
Application Workflow

From the user's point of view, a typical session flows through these stages:



Each arrow represents a hand-off with clear ownership: the UI collects data, the validator certifies it, the analytics service computes from it, generators only run on validated data, and every outcome (success or failure) is recorded.

Complete Data Flow



The same validated ReportData object feeds every downstream consumer — dashboard, comparison view and report generators — so a number can never differ between screens. This single-source-of-truth design is the backbone of the application's consistency.

UI Architecture

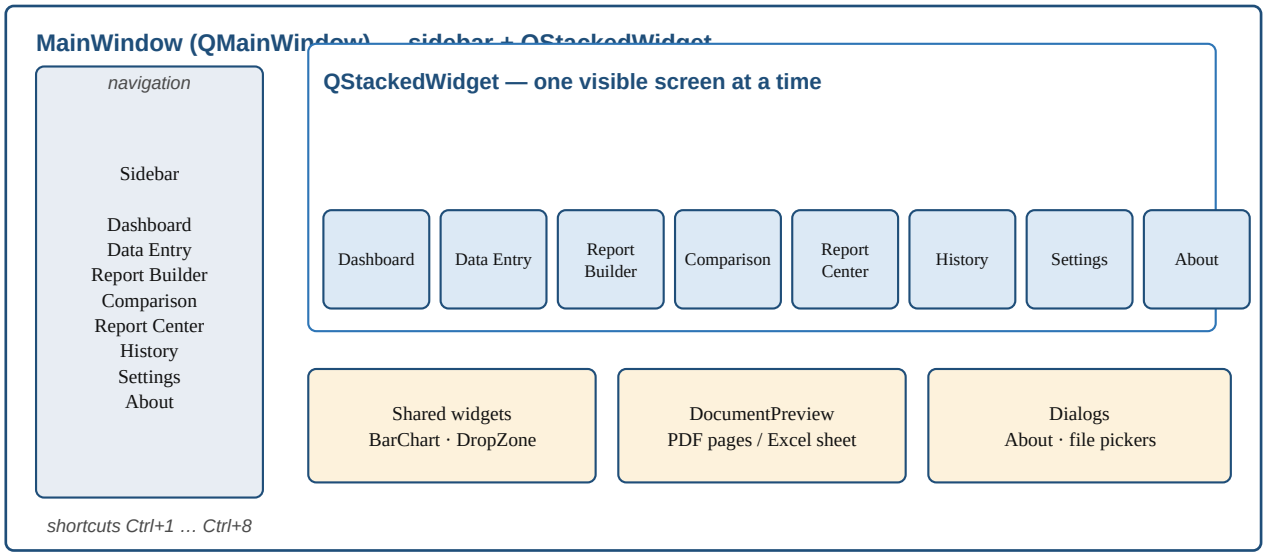


Figure 2 — UI architecture. MainWindow hosts a fixed sidebar and a QStackedWidget with the eight application screens; only one screen is visible at a time. Screens never reference each other directly — they communicate through the main window (e.g. 'generate report' switches to the Report Center with the validated dataset). Shared widgets are reused across screens.

Excel Processing

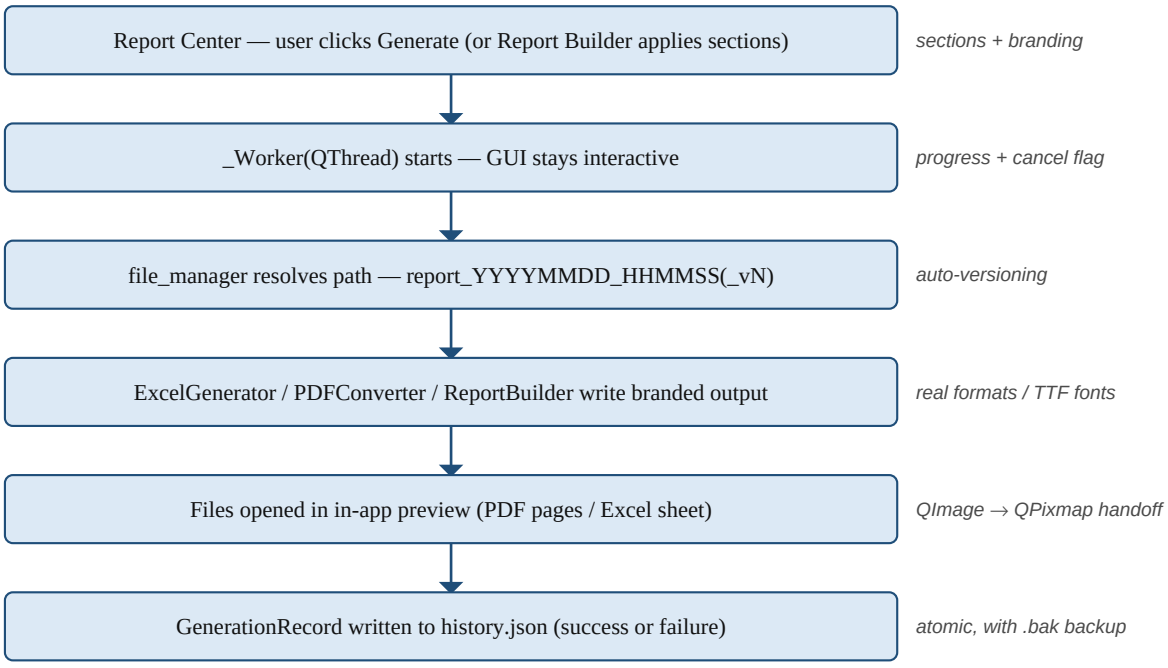
- **Import:** .xlsx files are parsed with openpyxl. Header detection inspects the first rows to find the header line, then maps columns flexibly — files with or without an ID column import correctly.
- **Header detection:** column names are matched case-insensitively against known headers (ID, Product, Category, Region, Month, Units, Price, Discount); unmatched columns are skipped safely.

- **Validation:** every imported row runs through the same validator as manual entry — nothing enters the dataset uncertified.
- **Data processing:** rows become SalesRecord objects; revenue and net revenue are derived properties, so they can never disagree with units × price.
- **Generation:** workbooks are produced with real Excel number formats (■#,##0.00, 0.0%), freeze panes, auto-filter and native openpyxl charts.
- **Formatting:** branded headers use the configured company colours and fonts; the detailed-sales sheet is section-aware in Report Builder output.
- **Preview:** the active worksheet is rendered inside the app so the user can inspect output without opening Excel.

PDF Processing

- **Generation:** PDFs are built with ReportLab's platypus layout engine — flowables stack into a multi-page document automatically.
- **Fonts:** TTF fonts are registered at runtime so the document is not limited to the base-14 PDF fonts.
- **■ support:** a font fallback chain prefers Windows fonts that contain the rupee glyph; rendering was explicitly verified so ■ appears in all currency output.
- **Preview:** PyMuPDF (fitz) renders each page to an image; pages are converted to QImage on the worker side and displayed via QPixmap on the GUI thread (a crash fix documented in the Improvements section).
- **Multi-page handling:** branded headers/footers repeat on every page with page numbers; long datasets paginate cleanly — the sample dataset produces a 2-page report verified by the integration test.

Generation flow (both formats):



Both formats share the same worker, cancellation and history machinery — only the document-writing step differs. This is why behaviour (progress, cancel, history entries) is identical for Excel and PDF output.

Analytics Engine

analytics.py is a pure, dependency-free module — the single source of truth for every number the application displays. Filtered subsets support all dashboard filters.

Group	Metrics provided
Revenue	total revenue, net revenue (after discounts), totals by month / category / region
Units	total units, units by month / category / region
Categories & Regions	breakdowns in calendar order, contribution percentages for categories and regions
Products	top/bottom N by revenue or units, product contribution percentages
Comparisons	compare_periods(): absolute difference + percentage change per metric; returns None (rendered as a dash) when the earlier period is empty
Insights	generate_insights(): deterministic statements with a <1% noise filter — best/worst periods, top movers, concentration warnings

Because the module is pure, every metric is covered by direct unit tests and behaves identically in the UI, in reports and in tests.

Threading Architecture

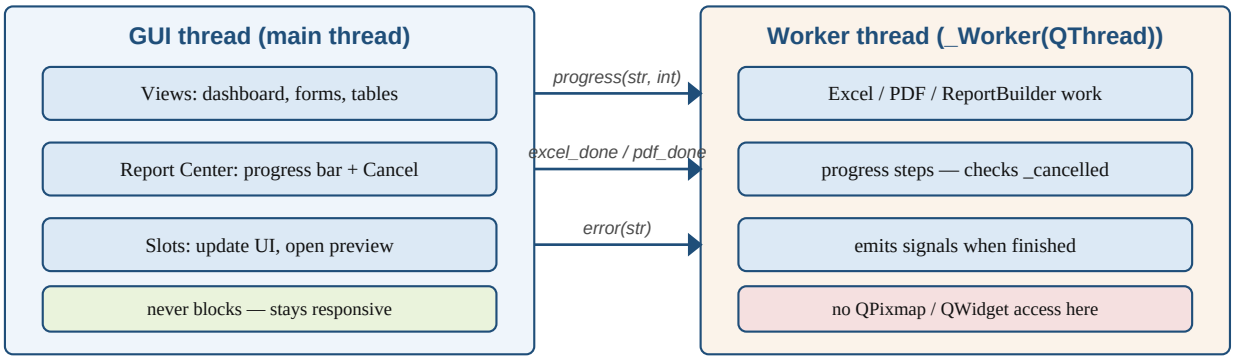


Figure 3 — Threading architecture. Heavy report generation runs on a _Worker(QThread) so the GUI thread never freezes. The worker publishes progress and results through Qt signals (progress, excel_done, pdf_done, error), which Qt delivers to slots on the GUI thread. Cancellation is cooperative: the GUI calls cancel(), the worker checks the flag at every step and exits cleanly — QThread.terminate() is never used, so files are never left half-written.

Data Models

Model	Fields	Purpose & relationships
SalesRecord (dataclass)	id, product_name, category, region, month, units_sold, unit_price, discount_pct — plus derived revenue and net_revenue properties	One row of sales data. Column metadata (COLUMNS, EDITABLE, READONLY) drives the data-entry table. Many records aggregate into one ReportData.
ReportData (dataclass)	records: List[SalesRecord], generated_at, title, subtitle — plus aggregate properties (count, total_revenue, total_net, total_units, total_discount, avg_discount) and by_category/by_region/by_month groupings	The full dataset passed to analytics and generators. Single source of truth shared by dashboard, comparison and reports.

Model	Fields	Purpose & relationships
GenerationRecord (dataclass)	id (uuid), timestamp (ISO), status (success/failure), records, excel path, pdf path, net_revenue, error, duration	One history entry. Serialises to/from JSON; time_str never crashes on malformed timestamps.
ValidationError / ValidationResult	row, col, msg — aggregated into ok / errors / warnings / summary	Validator output; drives cell-level feedback in the data-entry UI.
AppConfig / Branding / Settings	branding (company, colours, fonts, footer), settings (output dir, theme, auto-version, history limit), reference lists (categories, regions, months)	Single configuration source of truth, persisted atomically to the user config directory.

Configuration & Persistence

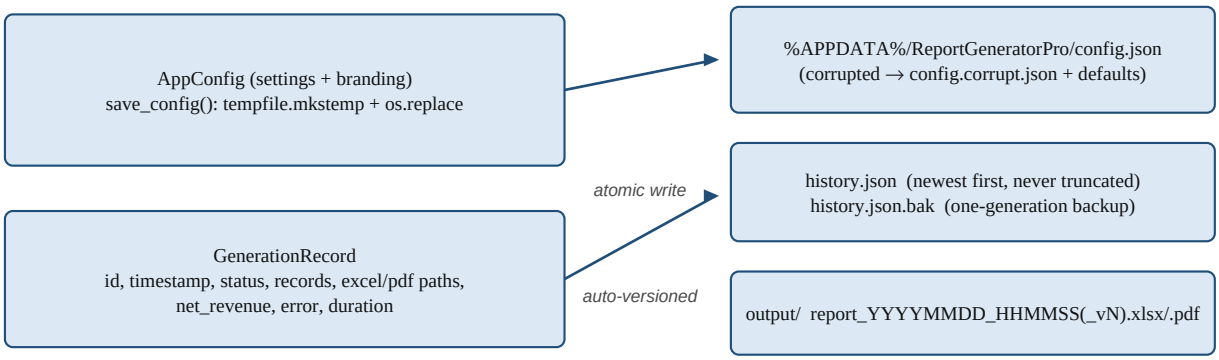


*Figure 4 — Configuration and history flow. Settings and generation history both persist as JSON with identical safety guarantees: write to a temp file, then os.replace() atomically. History keeps a one-generation .bak backup and never truncates records on disk; corrupted files are quarantined (renamed *.corrupt.json) instead of deleted, and the app falls back to defaults or an empty list rather than crashing.*

Validation & Error Handling

Validation rules (enforced per record by data_validator.py):

- Product name must be non-empty; category, region and month must match the configured reference lists.
- Units sold must be a non-negative integer; unit price must be a number greater than 0.
- Discount must be a number in 0–100; values above 50 produce a warning (not an error) asking the user to verify.
- An empty dataset is rejected with a clear message before any processing.

Error handling layers:

- **User errors** — cell-level messages in the data editor (row + column + reason), so problems are fixable in place.
- **Import errors** — malformed .xlsx files fail safely with an explanatory message; unmatched columns are skipped, not fatal.
- **Generation errors** — worker failures are captured via the error signal, shown in the UI, logged, and recorded as a failed entry in history with the error text.
- **Logging** — logger_service.py writes rotating technical log files to logs/ and keeps a user-facing activity log inside the app; main.py installs a global exception hook so unhandled errors are logged, never silently lost.

- **Recovery** — corrupted config/history files are quarantined and defaults applied; history keeps a .bak backup; the application always starts.

Security Considerations

This is an offline, single-user desktop application. The security model is local-file safety rather than network hardening — there is no network code, no API calls, no telemetry and no secrets handling.

- **Local file handling:** reads only user-selected .xlsx files via file dialogs/drop zones; reports and logs stay in the app's output/logs directories or the user-configured output directory.
- **Input validation:** all records pass the validation service before analytics or report generation; corrupted configuration cannot crash the application.
- **Secrets / environment variables:** no secrets, credentials or API keys are stored; only standard platform variables (APPDATA / XDG_CONFIG_HOME) are read.
- **File / path safety:** output paths are built by file_manager.py with automatic versioning; imports use header detection rather than trusting file structure; PDF preview renders pages to images via PyMuPDF.
- **Generated report handling:** plain .xlsx/.pdf documents in a user-controlled directory; history data keeps a .bak backup and is user-readable/deletable.

Security limitations (stated honestly): as a single-user offline tool the application does not implement authentication, encryption at rest or sandboxing — these are out of scope by design. Imported Excel files are parsed with openpyxl, which — like most parsers — should not be pointed at untrusted files.

Testing

Test structure:

- **tests/test_all.py** — 16 test classes covering models, validation, Excel/PDF generators, file manager, generation records, config persistence, history robustness, helpers, Excel import, path handling, font resolution, theme tokens and worker cancellation.
- **tests/test_analytics.py** — TestAnalytics, TestComparison, TestInsights (pure-metric coverage).
- **tests/test_builder.py** — TestReportBuilder, TestDemoData, TestActivityLog.
- **tests/conftest.py** — an isolated_env fixture redirecting config/history/output to temporary directories, so the real user's data is never touched by tests.

Test categories and edge cases:

- Unit tests (analytics, validation, generators, models), integration tests (data pipeline, import/export round-trips), and UI-level checks (screen construction, themes, cancellation).
- Edge cases: empty and single-record datasets, files with/without an ID column, empty comparison periods, corrupted configuration/history files, duplicate records, boundary discounts (0, 50, 100).

Actual test results (executed in this environment):

Verification	Command	Result
Unit/integration suite	python -m pytest tests -q	110 passed in 3.08s

Verification	Command	Result
End-to-end smoke test	python tests/verify_integration.py	ALL FINAL VERIFICATIONS PASSED — startup (8 screens), themes, validation, Excel + PDF generation on worker threads, history recording, Excel import round-trip, PDF preview threading, cooperative cancellation, dashboard filters/comparison/insights, report builder generation, activity log
Installation verification	pip install -r requirements.txt; python main.py	Application launches; all imports resolve; config/output directories created on first run
Packaging verification	build_windows.bat (PyInstaller)	dist\ReportGeneratorPro\ReportGeneratorPro.exe; resource paths resolve identically from source and packaged exe

Performance

Measured with tests/benchmark.py (actual output):

```
100 records  | validate 0.1 ms | analytics 0.7 ms | insights 0.8 ms | excel 0.03 s
1,000 records | validate 0.6 ms | analytics 6.6 ms | insights 7.6 ms | excel 0.19 s
5,000 records | validate 4.2 ms | analytics 38.7 ms | insights 39.1 ms | excel 0.73 s
10,000 records | validate 6.0 ms | analytics 65.0 ms | insights 76.3 ms | excel 1.60 s
```

- **Large datasets:** validation and analytics stay in the milliseconds even at 10,000 records because the analytics engine computes breakdowns in single passes.
- **Report generation performance:** Excel output for 10,000 records completes in 1.6 s on a worker thread, so the GUI remains fully interactive throughout.
- **Insights:** the deterministic insight engine scales linearly and remains sub-100 ms at 10,000 records.

Packaging & Deployment

- **PyInstaller:** report_generator_pro.spec + build_windows.bat produce a one-folder Windows build with a single command.
- **Executable:** dist\ReportGeneratorPro\ReportGeneratorPro.exe — no Python installation required on the target machine.
- **Required resources:** QSS theme files and fonts are bundled and resolved relative to the package, so the exe behaves identically to running from source.
- **Windows deployment:** verified end-to-end — the packaged exe starts, loads config, generates reports and writes history just like the development environment.

GitHub Readiness

Aspect	Status
Repository structure	Clean layout: app/ source, tests/, sample_data/, build script and spec at the top level
README	Full usage documentation: overview, features, architecture, installation, testing, sample data, packaging, limitations
.gitignore	Excludes __pycache__, .venv, dist/, build/, .pytest_cache, logs and generated output
requirements.txt	Pinned dependencies for reproducible installation
Tests in repo	Complete pytest suite + verification and benchmark scripts, runnable by any reviewer

Aspect	Status
Documentation	Architecture and data-pipeline descriptions in the README matching the actual code
No secrets / generated files	No credentials, keys or generated output committed; config and history live in user directories outside the repo

Problems Found & Improvements Made

Issues discovered during development were fixed and verified. Each is documented as Problem → Cause → Solution → Result:

Problem	Cause	Solution	Result
GUI freezes during report generation	Excel/PDF generation ran on the GUI thread	Moved generation onto QThread workers with progress signals and cooperative cancellation	UI stays responsive; long reports can be cancelled safely
QThread.terminate() risked corrupted output	Force-terminating threads mid-write can leave partial files	Replaced with a cooperative cancellation flag checked at safe points in the generation loop	No partial/corrupt reports; clean, deterministic cancellation
In-app PDF preview crashed intermittently	QPixmap (a GUI-native type) was created on a worker thread	Preview worker renders pages to QImage off-thread and converts to QPixmap on the GUI thread	Stable preview; covered by an integration check
Misleading percentages in period comparison	Division by an empty earlier period produced 0% or errors	compare_periods() returns None when the base period is empty; UI renders a dash	No misleading analytics reach the user or reports
■ symbol missing in PDF output	Default PDF fonts lack the rupee glyph	Registered TTF fonts with ReportLab, verified rendering, added a fallback chain	Currency renders correctly in all PDF reports
Corrupted config file broke startup	Config was read unguarded and written non-atomically	Atomic writes (temp file + os.replace) plus corruption quarantine and defaults fallback	Startup never fails due to configuration
Imports failed on files without an ID column	Import assumed a fixed column layout	Header detection and flexible column mapping during .xlsx import	Real-world spreadsheets import without manual editing
Fabricated-looking insights on small datasets	Insights fired on insignificant fluctuations	Noise filter suppresses changes below 1%	Insights are always meaningful and data-backed

Known Limitations

No critical or high-priority issues remain in the final implementation. The following limitations are genuinely present in the current build:

Priority	Limitation
Critical	None.
High	None.
Medium	History and activity logs are JSON files — appropriate for single-user desktop use, but not a concurrent datastore.

Priority	Limitation
Low	In-app PDF preview renders pages as images (no text selection/search within the preview).
Low	Windows-first font handling: on macOS/Linux the ■ glyph may degrade on exotic systems (DejaVu/Helvetica fallback is used).
Low	The bundled demo dataset is deterministic (for reproducible demos), not statistically realistic.

Future Improvements

- Interactive charts via Qt Charts (hover tooltips, zoom) building on the existing dashboard filters.
- Scheduled local report generation, integrating with Windows Task Scheduler using the existing pipeline.
- CSV export of the filtered dashboard view for ad-hoc analysis in other tools.
- Per-user Report Builder section presets, stored alongside existing config persistence.
- Optional SQLite backend for history if multi-process access ever becomes a requirement (replacing JSON storage behind the same API).
- Cross-platform font profile for Linux/macOS to make ■ rendering equally robust outside Windows.

Resume / Portfolio Value

Skill area	Evidence in this project
Python development	Python 3.10+, dataclasses, type hints, pathlib, packaging scripts
Desktop application development	Complete 8-screen product with settings, history, logging and accessibility support
PyQt5 UI development	Views, model/view tables, custom widgets, light/dark stylesheets, signals/slots
Object-oriented design	Layered modules, dataclass models, service classes, single-responsibility separation
File processing	Excel import with header detection and validation; JSON persistence with atomic writes and backups
Excel automation	openpyxl workbooks with native number formats, freeze panes, auto-filter and charts
PDF generation	ReportLab platypus layout, TTF font registration, branded headers/footers, page numbers
Multithreading	QThread workers, signals/slots, cooperative cancellation, thread-safe preview rendering
Error handling	Global exception hook, graceful fallbacks, rotating technical logs + user-facing activity log
Testing	110 pytest tests across unit/integration/UI categories, edge cases, end-to-end verification, benchmarking
Packaging	PyInstaller standalone Windows build verified from source and packaged exe
Git / GitHub practices	Repository with .gitignore, README, requirements.txt, sample data and reproducible test/build commands

Beyond the skill list, the project demonstrates engineering judgement: choosing cooperative cancellation over terminate(), making analytics pure so it is testable, designing for corrupted-state recovery, and measuring performance instead of guessing. These habits transfer directly to production software teams.

Final Technical Summary

Dimension	Summary
Architecture	Five-layer modular design (UI → models → services → utilities/config → storage) with strict separation of concerns and single-source-of-truth data flow
Technologies	Python 3.10+, PyQt5, openpyxl, ReportLab, PyMuPDF, pytest, JSON, PyInstaller
Features	Data entry/import, analytics dashboard, insights, section-based report builder, branded Excel/PDF output, in-app previews, history, themes, accessibility
Data flow	Input → validation → models → analytics → reports → storage/history, with one validated ReportData object feeding every consumer
Testing	110 passing tests + end-to-end verification script + performance benchmark, all executed and verified
Packaging	PyInstaller build producing a standalone Windows executable, verified from source and packaged exe
Current status	Complete and verified: no critical or high-priority issues remain; all quality checks pass in the final build

---

All figures in this report (test counts, benchmark timings, verification results) were produced by actually executing the test suite, verification script and benchmark in the development environment.